CSS Grid Order Property

Complete Guide: From Basics to Advanced

Contents

Chapter 1

Introduction

1.1 What is the Order Property?

The `order` property in CSS Grid controls the visual order of grid items without changing the HTML structure. It allows you to reorder elements purely with CSS, making it powerful for responsive designs where you might want items to appear in different sequences on different screen sizes.

Key Point: The `order` property only affects visual order, not the DOM order. Screen readers and keyboard navigation still follow the HTML structure.

1.2 Order vs. Position

Order Property	**Position Property**
Reorders grid items	Moves elements in space
Doesn't affect layout flow	Takes element out of flow
Simple and clean	More complex positioning
Only works in flexbox/-grid	Works everywhere
Non-intrusive to DOM	Can overlap elements

Chapter 2

Core Order Properties

2.1 The order Property

2.1.1 Basic Syntax

```
.grid-item {
    order: <integer>;
}
```

2.1.2 Values Explained

- **Default:** `0`
- **Type:** Integer (positive or negative)
- **Range:** Any integer value (commonly -1000 to 1000)
- **Inheritance:** No

2.1.3 How It Works

1. All grid items have a default `order` value of `0`
2. Items are arranged from lowest order value to highest
3. Items with the same order value appear in source order
4. Negative values place items before default items

2.1.4 Examples

Example 1: Simple Reordering

```
.grid {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 10px;
}
```

```
.item-1 { order: 3; }  /* Appears third */
.item-2 { order: 1; }  /* Appears first */
.item-3 { order: 2; }  /* Appears second */
```

Result: Items appear in order 2, 3, 1 (not 1, 2, 3)

Example 2: Using Negative Values

```
.grid-item-featured {
    order: -1;  /* Appears before all default items */
}

.grid-item-normal {
    order: 0;   /* Default position */
}

.grid-item-last {
    order: 1;   /* Appears after default items */
}
```

Example 3: Mobile-First Responsive

```
/* Desktop layout */
.header { order: 1; }
.sidebar { order: 2; }
.main { order: 3; }
.footer { order: 4; }

/* Mobile layout - reorder items */
@media (max-width: 768px) {
    .header { order: 1; }  /* Stays first */
    .main { order: 2; }    /* Move main up */
    .sidebar { order: 3; } /* Sidebar after main */
    .footer { order: 4; }  /* Footer stays last */
}
```

Chapter 3

Real-World Use Cases

3.1 Use Case 1: Responsive Navigation

Reorder navigation items based on screen size.

```
.nav {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(100px, 1fr));
    gap: 10px;
}

.nav-logo { order: 1; }
.nav-links { order: 2; }
.nav-search { order: 3; }
.nav-user { order: 4; }

/* Mobile: Logo first, search/user at bottom */
@media (max-width: 600px) {
    .nav-search { order: 5; }
    .nav-user { order: 6; }
}
```

3.2 Use Case 2: Article with Sidebar

```
.article-container {
    display: grid;
    grid-template-columns: 1fr 300px;
    gap: 20px;
}

/* Desktop: Article on left, sidebar on right */
.article { order: 1; }
.sidebar { order: 2; }

/* Mobile: Sidebar moves below article */
@media (max-width: 768px) {
    .article-container {
        grid-template-columns: 1fr;
```

```
    }
    .article { order: 1; }
    .sidebar { order: 2; }
}
```

3.3 Use Case 3: Feature Highlight

Move an important item to the top without changing HTML.

```
.product-grid {
    display: grid;
    grid-template-columns: repeat(4, 1fr);
    gap: 15px;
}

.product { order: 0; }

/* Highlight featured product */
.product.featured {
    order: -1;  /* Appears first */
    grid-column: 1 / 2;
}
```

3.4 Use Case 4: Footer Stacking

```
.footer {
    display: grid;
    grid-template-columns: repeat(4, 1fr);
    gap: 20px;
}

.footer-copyright { order: 1; }
.footer-links { order: 2; }
.footer-social { order: 3; }
.footer-contact { order: 4; }

/* Mobile: Reverse order */
@media (max-width: 600px) {
    .footer {
        grid-template-columns: 1fr;
    }
    .footer-copyright { order: 4; }
    .footer-contact { order: 1; }
    .footer-social { order: 2; }
    .footer-links { order: 3; }
}
```

Chapter 4

Common Mistakes & Solutions

4.1 Mistake 1: Forgetting About Accessibility

Problem: Using order to reorder content in a way that doesn't match the HTML sequence causes confusion for screen reader users.

Solution: Always keep the HTML structure logical. Use order only for visual adjustments that make sense.

```
/* BAD: Completely changes meaning */
.item-1 { order: 10; }
.item-2 { order: 1; }
.item-3 { order: 5; }

/* GOOD: Minor visual adjustments */
.item-featured { order: -1; }
.item-normal { order: 0; }
```

4.2 Mistake 2: Using Very Large Order Values

Problem: Using huge numbers makes code hard to understand and maintain.

Solution: Keep values small and relative. Use -1, 0, 1 or similar.

```
/* BAD */
.item { order: 999999; }

/* GOOD */
.item { order: 1; }
```

4.3 Mistake 3: Confusing order with z-index

Problem: order changes visual position, not stacking. For layering, use z-index.

```
/* order controls position in flow */
.item { order: 2; }

/* z-index controls layering (overlapping) */
.item { z-index: 10; }
```

Chapter 5

Browser Support

5.1 Current Support

The `order` property is fully supported in all modern browsers:

- Chrome 29+
- Firefox 28+
- Safari 9+
- Edge 12+
- Opera 12.1+
- IE 11+ (partial support)

5.2 Vendor Prefixes

Modern browsers don't require prefixes. For older projects:

```
.grid-item {
    -webkit-order: 1;  /* Safari, old Chrome */
    -moz-order: 1;     /* Firefox */
    -ms-order: 1;      /* IE 10 */
    order: 1;          /* Standard */
}
```

Chapter 6

Advanced Techniques

6.1 Dynamic Order with CSS Variables

```
.grid-item {
    order: var(--item-order, 0);
}

/* Change with media queries */
@media (max-width: 768px) {
    .grid-item:nth-child(1) { --item-order: 2; }
    .grid-item:nth-child(2) { --item-order: 1; }
    .grid-item:nth-child(3) { --item-order: 3; }
}
```

6.2 Order with Animation

```
.grid-item {
    order: 0;
    transition: order 0.3s ease;
}

.grid:hover .grid-item:nth-child(2) {
    order: 1;
}

.grid:hover .grid-item:nth-child(1) {
    order: 2;
}
```

6.3 Combining with Grid Auto-Flow

```
.smart-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
    grid-auto-flow: dense;  /* Fill gaps automatically */
```

```
    gap: 10px;
}

.featured-item {
    grid-column: span 2;
    grid-row: span 2;
    order: -1;  /* Ensure it appears first */
}
```

Chapter 7

Summary: Key Takeaways

7.1 When to Use order

- Responsive layouts that need different item sequences
- Highlighting featured items without changing HTML
- Mobile-first design patterns
- Emphasizing important content
- Reordering navigation elements

7.2 When NOT to Use order

- For fundamental layout changes (restructure HTML instead)
- When it contradicts the logical HTML structure
- For stacking/layering (use z-index instead)
- To hide elements (use display: none or visibility)

7.3 Properties Recap

Property	Values	Default
order	`<integer>`	`0`

7.4 Quick Rules

1. Default order value is `0`
2. Lower values appear first
3. Negative values work fine
4. Same order values follow HTML source order
5. Always maintain logical HTML structure

6. Test with keyboard and screen readers
7. Keep values simple (not 999999)
8. Remember order doesn't affect z-index

7.5 Code Template

```
/* Basic responsive reordering */
.grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
    gap: 20px;
}

.item-primary { order: 1; }
.item-secondary { order: 2; }
.item-tertiary { order: 3; }

@media (max-width: 768px) {
    .item-primary { order: 1; }
    .item-tertiary { order: 2; }
    .item-secondary { order: 3; }
}
```